

TP n°5 — Sessions en PHP

Mécanisme d'authentification PHP & MySQL

Élève : Mohammed Fahad

Classe : BTS SIO 1ère année — SISR

Date : 5 mai 2026

1. Introduction

L'objectif de ce travail pratique est de mettre en place un mécanisme d'authentification en PHP, en utilisant les sessions et une base de données MySQL. Ce type d'authentification est une base incontournable du développement web.

1.1 Objectifs

- Créer et tester localement un site d'authentification PHP.
- Utiliser les sessions PHP pour maintenir l'état de connexion d'un utilisateur.
- Interroger une base de données MySQL pour vérifier les identifiants saisis.
- Afficher le nom et prénom de l'utilisateur connecté sur la page d'accueil.
- Améliorer la gestion des erreurs (login inconnu / mot de passe incorrect).

1.2 Environnement de travail

Élément	Détail
Serveur local	XAMPP (Apache + MySQL + PHP)
Répertoire	htdocs/sessions/
Base de données	monsite
Table principale	utilis
Langage	PHP 7/8 + MySQL
URL locale	http://localhost/sessions/

2. Base de données

La base de données monsite contient une seule table utilis qui stocke les informations des utilisateurs autorisés à se connecter.

2.1 Structure de la table utilis

Champ	Type	Contrainte	Rôle
NumUtil	INT(11)	PK / AUTO_INCREMENT	Identifiant unique
NomUtil	VARCHAR(30)	NOT NULL	Nom de famille
PrenomUtil	VARCHAR(30)	NOT NULL	Prénom
LoginUtil	VARCHAR(10)	NOT NULL	Identifiant de connexion
PassUtil	VARCHAR(10)	NOT NULL	Mot de passe

2.2 Script SQL

Le script ci-dessous crée la base, la table et insère deux utilisateurs de test.

```
CREATE DATABASE IF NOT EXISTS monsite CHARACTER SET utf8 COLLATE utf8_unicode_ci;
USE monsite;
```

```
GRANT ALL PRIVILEGES ON monsite.* TO 'monsite_util'@'localhost' IDENTIFIED BY 'secret';
```

```
DROP TABLE IF EXISTS utilis;
CREATE TABLE utilis (
  NumUtil integer NOT NULL AUTO_INCREMENT,
  NomUtil varchar(30) NOT NULL,
  PrenomUtil varchar(30) NOT NULL,
  LoginUtil varchar(10) NOT NULL,
  PassUtil varchar(10) NOT NULL,
  PRIMARY KEY (NumUtil)
) ENGINE=InnoDB CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

```
INSERT INTO utilis(NomUtil, PrenomUtil, LoginUtil, PassUtil)
VALUES ('Alizan', 'Gaspar', 'galiz', 'azerty');
```

```
INSERT INTO utilis(NomUtil, PrenomUtil, LoginUtil, PassUtil)
VALUES ('Diossy', 'Daisy', 'ddios', '12345');
```

Note : en production, les mots de passe ne doivent jamais être stockés en clair. Il faut utiliser password_hash() / password_verify().

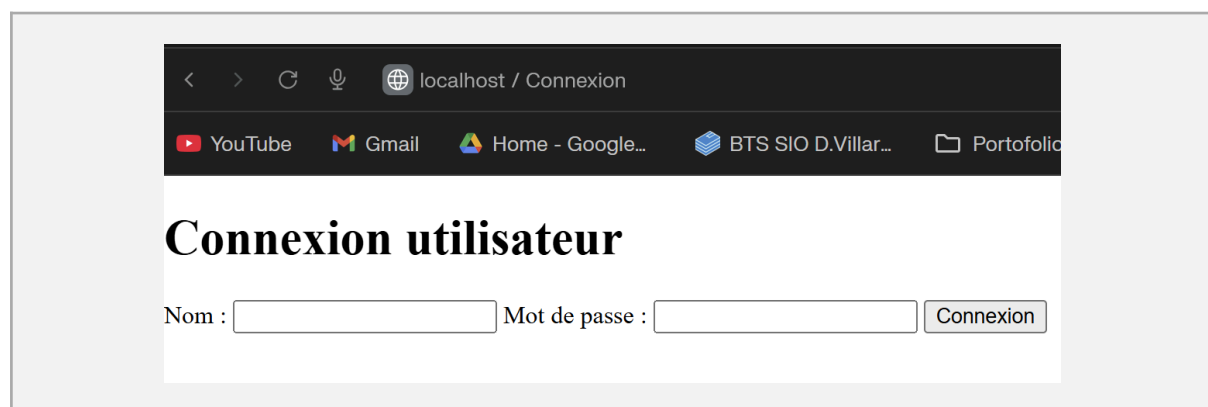
Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ utilis	Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8_unicode_ci	16,0 kio	-
1 table	Somme	2	InnoDB	utf8_unicode_ci	16,0 kio	0 o
☐ Tout cocher Avec la sélection :						

3. Fichiers PHP

3.1 login.php — Formulaire de connexion

Ce fichier affiche un formulaire permettant à l'utilisateur de saisir son identifiant et son mot de passe. Les données sont envoyées en POST vers login_post.php.

```
<!doctype html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Connexion</title>
</head>
<body>
    <h1>Connexion utilisateur</h1>
    <form action="login_post.php" method="post">
        <label for="nom">Nom :</label>
        <input type="text" name="login" id="nom" required />
        <label for="mdp">Mot de passe :</label>
        <input type="password" name="mdp" id="mdp" required />
        <input type="submit" value="Connexion">
    </form>
</body>
</html>
```



3.2 fonctions.php — Connexion PDO

Ce fichier contient la fonction `getBdd()` qui retourne une connexion PDO à la base monsite. Il est inclus dans les pages qui en ont besoin via `require`.

```
<?php
function getBdd() {
    try {
        $pdo = new PDO(
            'mysql:host=localhost;dbname=monsite;charset=utf8',
            'monsite_util',
            'secret',
            [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
        );
        return $pdo;
    } catch (PDOException $e) {
        die('Erreur de connexion : ' . $e->getMessage());
    }
}
?>
```

3.3 login_post.php — Traitement de l'authentification

Ce fichier gère la soumission du formulaire. Il démarre une session, effectue une requête SQL préparée pour vérifier les identifiants, puis enregistre les données en session si l'utilisateur est reconnu.

Les requêtes préparées (PDO prepare + execute) protègent contre les injections SQL — bonne pratique essentielle en développement web sécurisé.

```
<?php
session_start();
$erreur = '';

if (isset($_POST['login']) && isset($_POST['mdp'])) {
    require 'fonctions.php';
    $bdd = getBdd();

    // Étape 1 : vérifier si le login existe
    $req1 = $bdd->prepare('SELECT * FROM utilis WHERE LoginUtil=?');
    $req1->execute([$_POST['login']]);
    $userLogin = $req1->fetch();

    if (!$userLogin) {
```

```

        $erreur = 'Login inconnu. Vérifiez votre identifiant.';
    } else {
        // Étape 2 : vérifier le mot de passe
        $req2 = $bdd->prepare('SELECT * FROM utilis WHERE LoginUtil=? AND
PassUtil=?');
        $req2->execute([$_POST['login'], $_POST['mdp']]);
        $userOK = $req2->fetch();

        if ($userOK) {
            $_SESSION['login'] = $userOK['LoginUtil'];
            $_SESSION['nom'] = $userOK['NomUtil'];
            $_SESSION['prenom'] = $userOK['PrenomUtil'];
            header('Location: index.php');
            exit;
        } else {
            $erreur = 'Mot de passe incorrect.';
        }
    }
}
?>
<!doctype html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Résultat de l'authentification</title>
</head>
<body>
    <h1>Résultat de l'authentification</h1>
    <?php if ($erreur != '') { ?>
        <p style="color:red;"><?php echo $erreur; ?></p>
        <a href="login.php">Nouvel essai</a>
    <?php } ?>
</body>
</html>

```

Connexion utilisateur

Nom : Mot de passe :

Not Found

The requested URL was not found on this server.

Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at localhost Port 80

Connexion utilisateur

Nom :

Mot de passe :

Résultat de l'authentification

Login inconnu. Vérifiez votre identifiant.

[Nouvel essai](#)

3.4 index.php — Page d'accueil

La page d'accueil vérifie que l'utilisateur est connecté. Si ce n'est pas le cas, il est redirigé vers le formulaire. Elle affiche le nom et le prénom issus de la session.

```
<?php
session_start();
if (!isset($_SESSION['login'])) {
    header('Location: login.php');
    exit;
}
?>
<!doctype html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Accueil</title>
</head>
<body>
    <h1>Bienvenue</h1>
    <p>
        Bonjour <?php echo htmlspecialchars($_SESSION['prenom']); ?>
        <?php echo htmlspecialchars($_SESSION['nom']); ?> !
    </p>
    <a href="logout.php">Se déconnecter</a>
</body>
</html>
```

Bienvenue

Bonjour Gaspar Alizan !

[Se déconnecter](#)

6. Conclusion

Ce TP a permis de mettre en œuvre un mécanisme d'authentification complet en PHP avec gestion de sessions et accès sécurisé à une base de données MySQL via PDO. Les sessions PHP constituent un outil fondamental pour maintenir l'état d'un utilisateur d'une page à l'autre.

Points retenus

- Utilisation de `session_start()`, `$_SESSION` et gestion du cycle de vie d'une session.
- Requêtes préparées PDO pour se protéger des injections SQL.
- Redirections PHP avec `header('Location: ...')` et `exit`.
- Distinction des cas d'erreur lors de l'authentification.
- Utilisation de `htmlspecialchars()` pour prévenir les failles XSS.

Pistes d'amélioration

- Hacher les mots de passe avec `password_hash()` / `password_verify()`.
- Implémenter une déconnexion propre via `session_destroy()`.
- Limiter les tentatives de connexion (protection brute force).
- Passer le site en HTTPS pour chiffrer les données en transit.